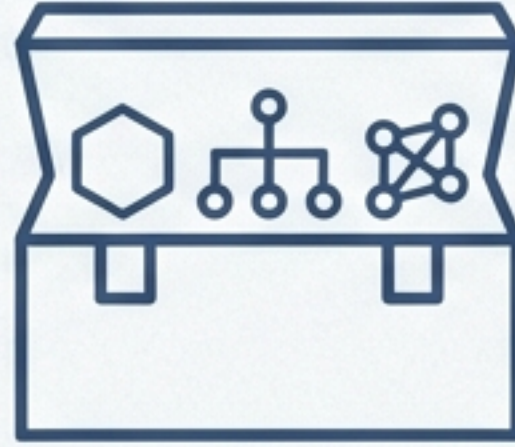




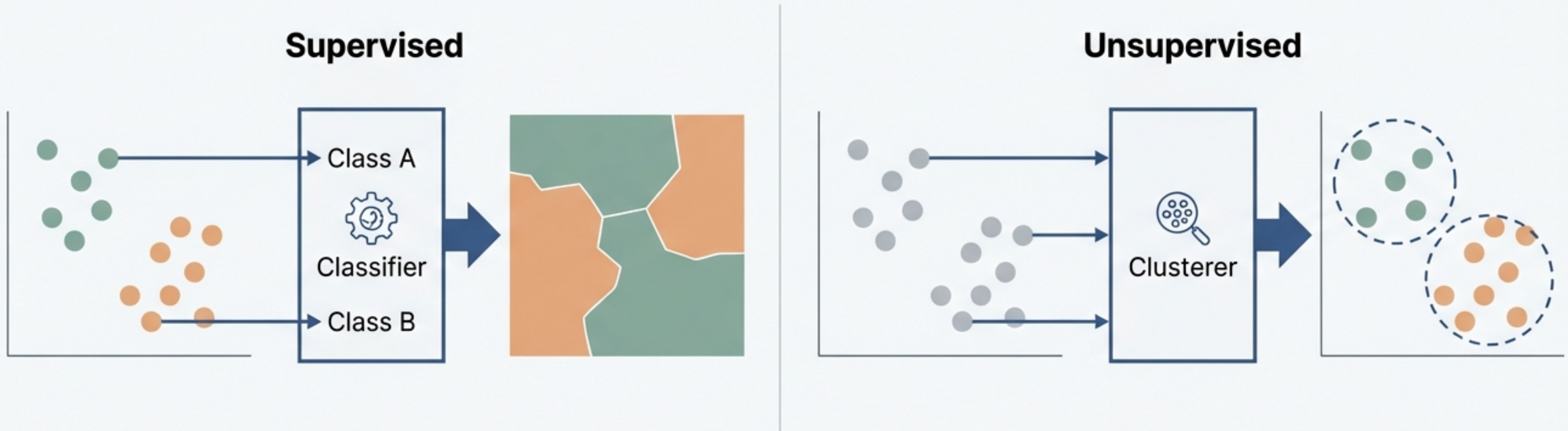
A Specialist's Toolbox for Unsupervised Classification in Google Earth Engine

A Practical Guide to GEE's Five Clustering Algorithms

From Supervised to Unsupervised: Classification Without Labels

In many cases, you may not have a large list of labelled training data. Unsupervised classification, or **clustering**, empowers an algorithm to discover the underlying patterns and figure out different land cover classes on its own.

The process involves looking at a large number of pixels, determining which are most similar, and grouping them into distinct “clusters”.



The Clustering Workflow in Earth Engine



1. Select Image & ROI

Start with the image and region of interest you want to classify.

2. Generate Samples

Create training data. Unlike supervised methods, this data does not need class labels. A simple random sample of pixels is sufficient to start. (e.g., ``image.sample()``)

3. Train the Clusterer

Instantiate a Clusterer object and train it using the ``train()`` function on your sample data.

4. Apply to Image

Apply the trained clusterer to the full image to generate the final output, where each pixel is assigned to a cluster.

Inside the Toolbox: The Five WEKA Clusterers

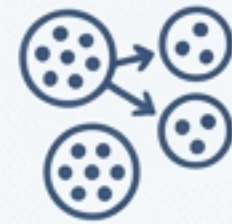
Google Earth Engine integrates the powerful WEKA library, a collection of tools for data analysis and data mining developed in New Zealand. This provides access to five distinct clustering algorithms, each with a unique approach to grouping data.



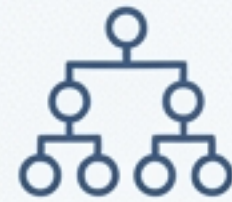
`ee.Clusterer.wekaKMeans`



`ee.Clusterer.wekaCascadeKMeans`



`ee.Clusterer.wekaXMeans`



`ee.Clusterer.wekaCobweb`

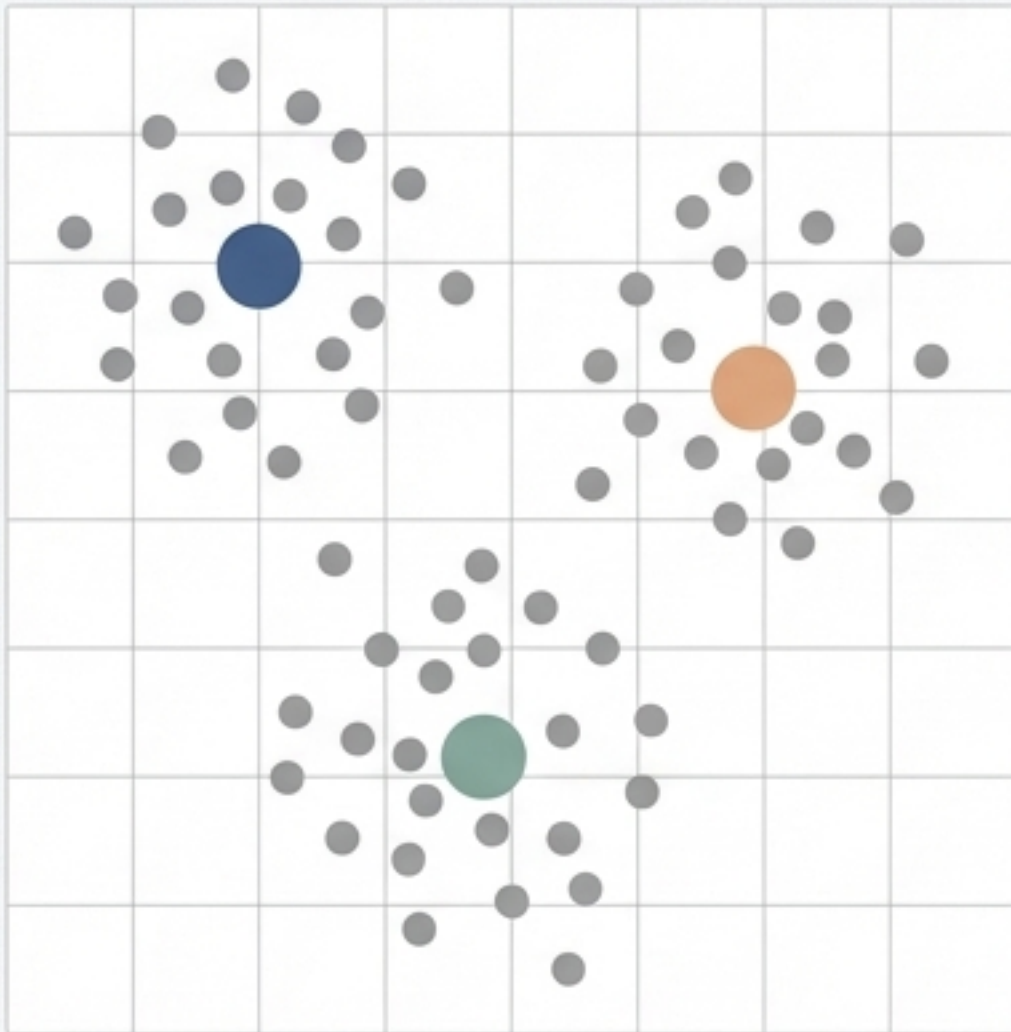


`ee.Clusterer.wekaLVQ`

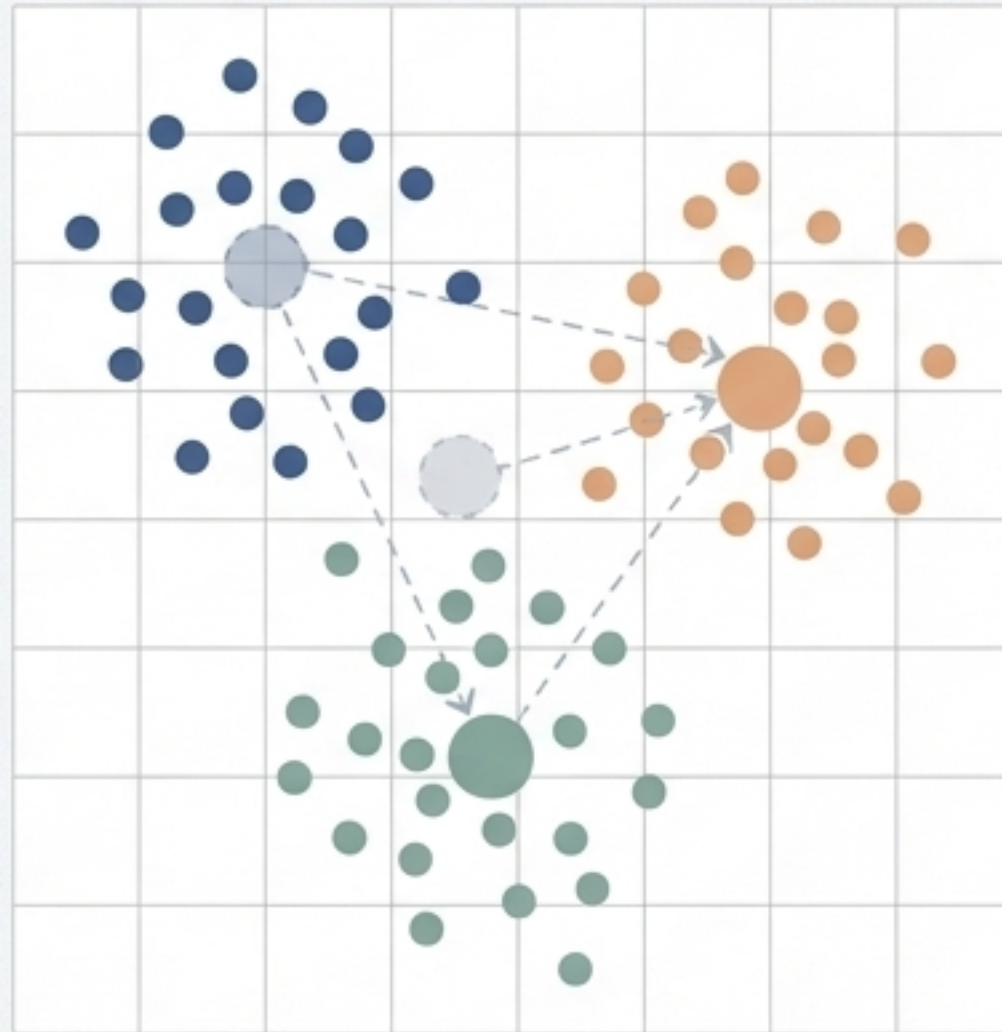
The Foundation: Understanding K-Means

K-Means partitions data into a pre-determined number of clusters ('k'). It is an iterative algorithm that aims to minimise the distance between points within the same cluster.

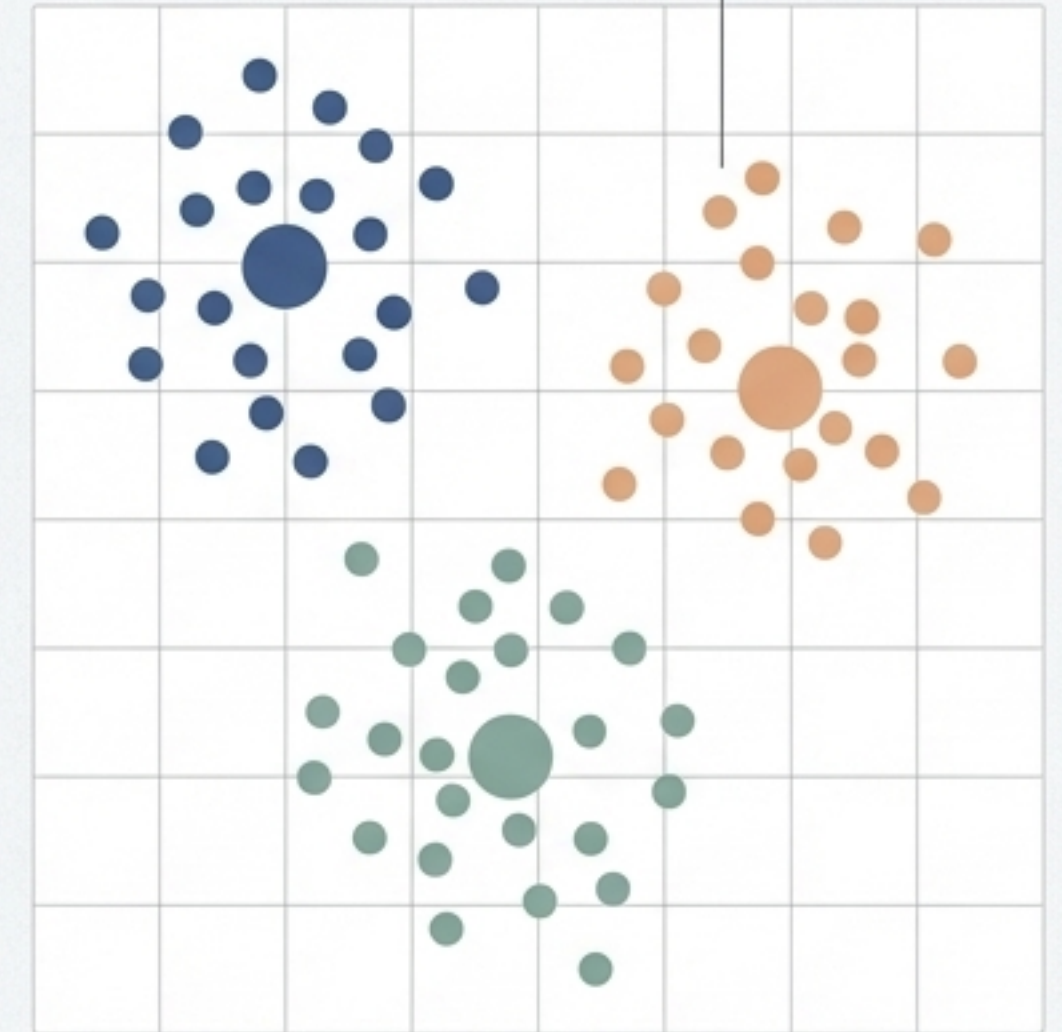
Initialise



Assign & Update



Converge



The algorithm stops when the cluster assignments no longer change, a state known as convergence.

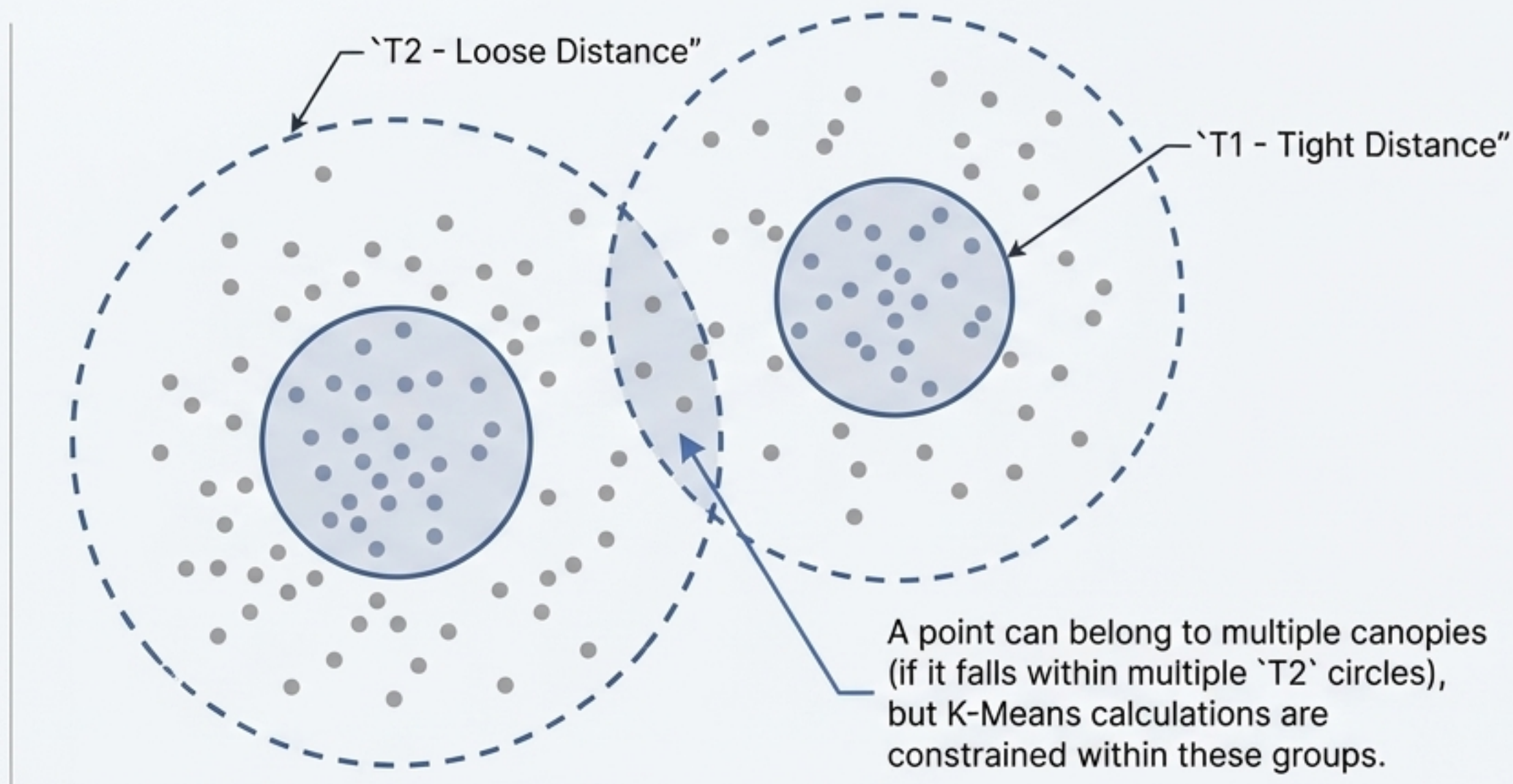
Tuning K-Means: Key Parameters

Core Settings		Performance & Termination	
numberOfClusters	**Required.** The number of clusters (`k`) to create.	maxIterations	An upper limit on iterations if convergence is not reached.
init	Method for placing initial cluster centres. Options: `random` (default), `k-means++` (often faster convergence), `canopy`, `farthest_points`.	fast	Trades numerical precision for improved performance; disables error metrics.
distanceFunction	The metric used for distance calculation. Options: `Euclidean` or `Manhattan`.	Canopy Options	
		useCanopies	A boolean to enable Canopy Clustering for significant performance gains on large datasets. (Note: 'See next slide for details.')
		Reproducibility	
		seed	A number to ensure the 'random' aspects of the algorithm are repeatable.

Note on `preserveOrder`: The effect of this parameter is not clearly described in the documentation. It is best left at its default unless you have a specific reason to change it.

Accelerating K-Means with Canopy Clustering

Canopy clustering is a **pre-clustering technique** that dramatically improves K-Means performance. It groups points into overlapping “canopies”, and the main algorithm then only compares distances between points that fall within the same canopy.



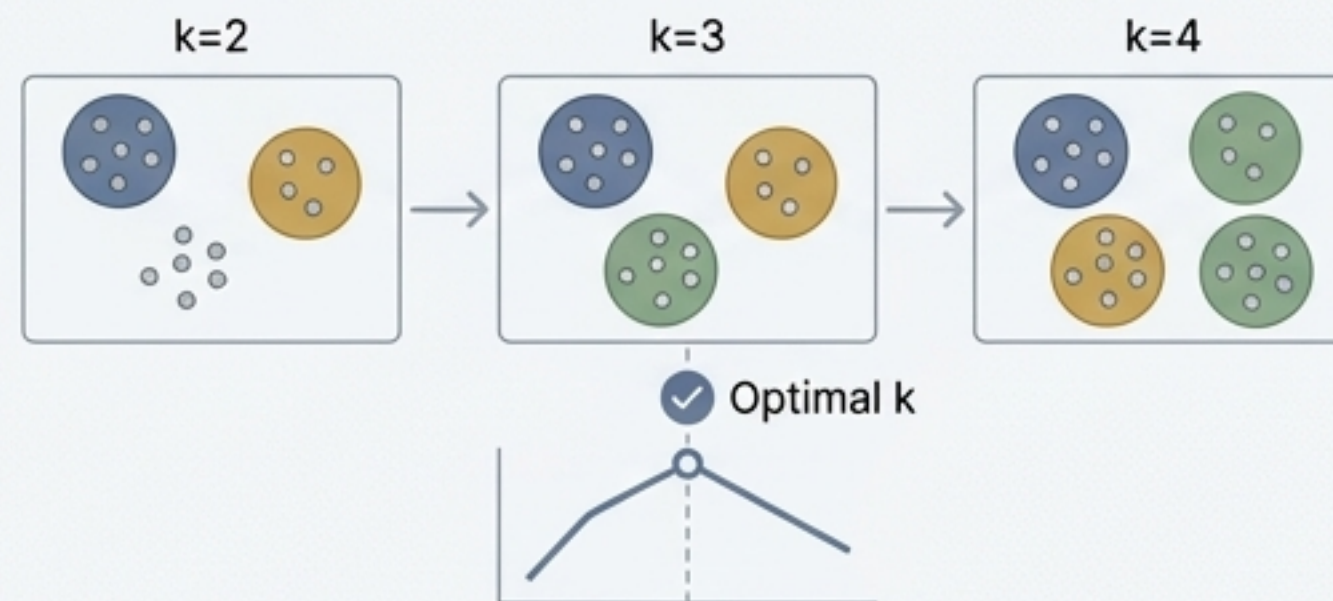
Key Parameters

- **t1**: The “tight distance”. Canopy centres must be further apart than this distance.
- **t2**: The “loose distance”. All points within this distance of a canopy centre are included in that canopy. (**t2** > **t1**) by definition).

The K-Means Family: Automating Cluster Detection

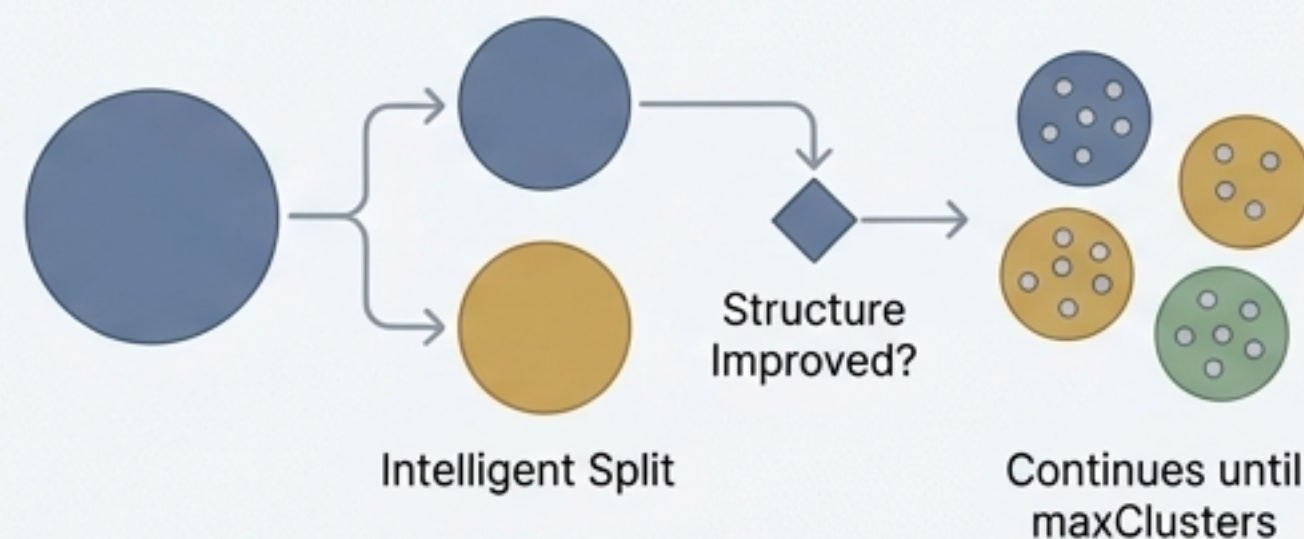
1. Cascade K-Means (wekaCascadeKMeans)

- **Core Function:** An extension of K-Means that automatically determines the optimal number of clusters.
- **Mechanism:** It iterates over a range of cluster counts (from `minClusters`` to `maxClusters``) and selects the count that best satisfies the "Variance Ratio Criterion"—maximising the distinction between clusters while minimising variation within them.



2. X-Means (wekaXMeans)

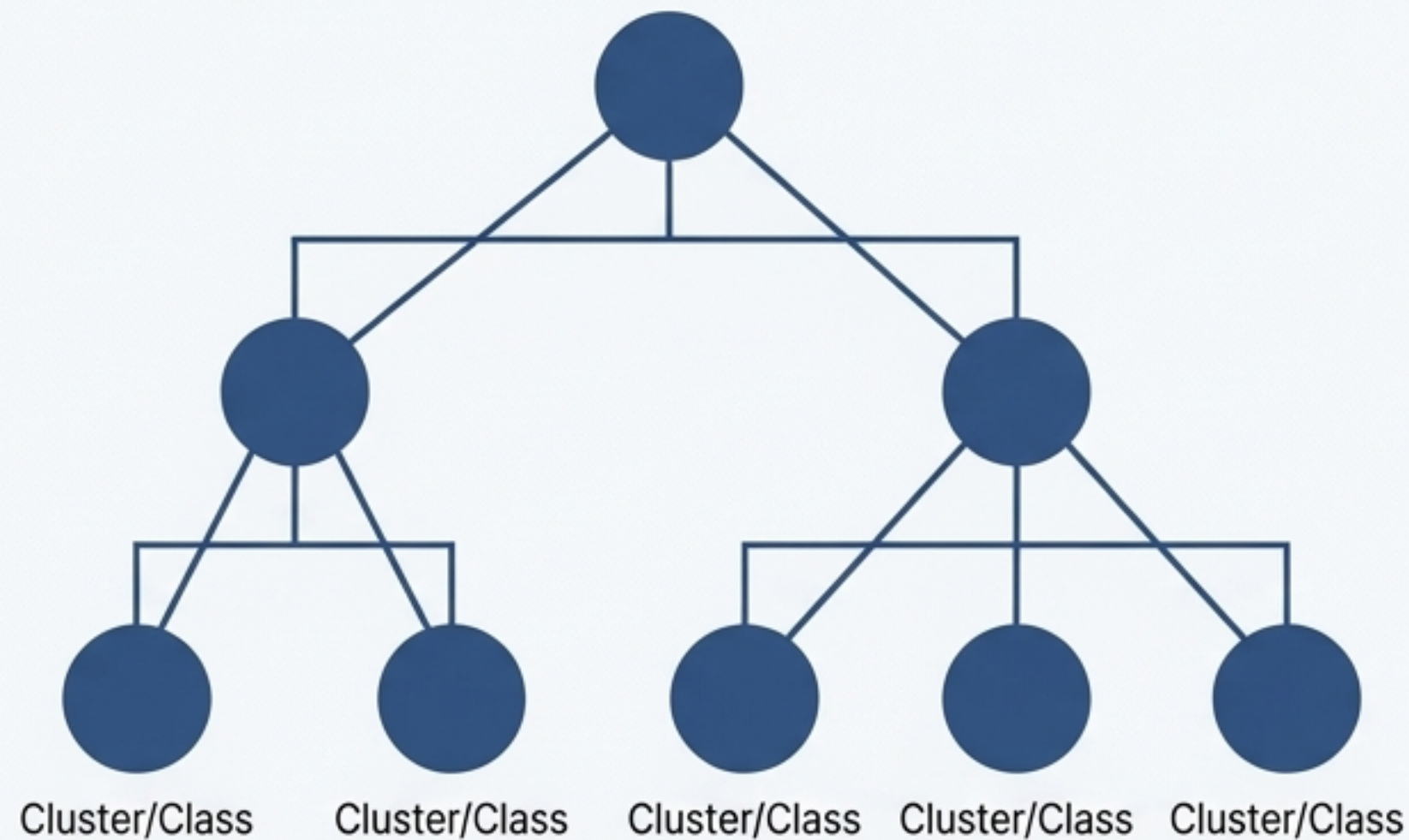
- **Core Function:** Another K-Means variant that automatically finds a suitable number of clusters.
- **Mechanism:** It begins with an initial number of clusters (`minClusters`) and intelligently splits the most promising clusters, keeping the split, if it improves the overall structure. This process continues until `maxClusters`` is reached.



Both algorithms remove the guesswork of choosing ``k``, but require you to define a search range instead. 🔍

A Different Approach: Cobweb's Classification Tree

Cobweb is a conceptual clustering algorithm. Instead of partitioning points, it organises them into a hierarchical classification tree, where each node represents a class or concept.



Controlling the Tree Structure

acuity: Controls the sharpness of category definitions. **Increasing acuity** makes nodes less likely to split, resulting in a tree with fewer nodes (and therefore fewer final clusters).

cutoff: Sets the minimum required 'category utility' or goodness for a split to occur. **Increasing the cutoff** demands more improvement from a split, also resulting in fewer branches and fewer clusters.

The Neural Network Tool: Learning Vector Quantization (LVQ)

Core Principle: LVQ is a neural network-based algorithm that uses a competitive learning strategy, similar in spirit to K-Means.

Mechanism: During each iteration (or 'epoch'), the algorithm loops through all training points. For each point, it finds the closest cluster centre and 'pulls' that centre slightly towards the point's location.

Key Parameters

- `numClusters`: The desired number of clusters.
- `learningRate`: Determines how strongly the cluster centre is pulled towards a training point in each step.
- `epochs`: The number of times the algorithm iterates through the entire training dataset.
- `normalizeInput`: **Important.** A boolean to normalise input band values. Should be used if input bands have significantly different ranges (e.g., 0-1 vs. 0-100). Default is `false`, assuming most satellite bands have similar ranges.

The Specialist's Blueprint: Choosing Your Algorithm

Algorithm	Core Principle	Requires `k`?	Key Strength / Best Use Case
K-Means	Iterative partitioning to minimise intra-cluster distance.	Yes	Simple, fast with canopies, and a good general-purpose starting point when you know the number of classes.
Cascade K-Means	Iterates K-Means over a range of `k` to find the optimum based on variance.	No (Range)	Automatically finds the best number of clusters based on statistical criteria.
X-Means	Iteratively splits clusters to find an improved structure.	No (Range)	Another powerful method for discovering the number of clusters automatically, using a structural approach.
Cobweb	Builds a hierarchical classification tree from the data.	No	Creates a conceptual hierarchy, which can be useful for understanding nested relationships between classes.
LVQ	A neural network that 'pulls' cluster centres towards data points.	Yes	A machine learning approach that can be effective, especially when fine-tuning `learningRate` and `epochs`.

Mastering the Toolbox: A Concluding Perspective

There is no single 'best' clustering algorithm. The optimal choice depends entirely on your data, your computational resources, and your specific analytical goals.



Key Takeaways

- **Start with K-Means:** If you have a hypothesis about the number of classes, K-Means is a direct and understandable starting point. Remember to use canopies for large datasets.
- **Automate When Unsure:** Use Cascade or X-Means when the number of clusters is unknown and you want the algorithm to guide your discovery.
- **Explore Different Models:** Consider Cobweb or LVQ when standard partitioning methods do not yield satisfactory results and you want to explore fundamentally different approaches.

The most effective strategy is to experiment. Try different algorithms and tune their parameters to discover which tool in the toolbox works best for your application.